ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ

Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Үйлдлийн систем (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр:	UNIX Shell
Вариант:	№1
Хичээл заасан багш:	Г. Ганбат
Бие даалтын ажлыг гүйцэтгэсэн:	Оюутан: B231940067 (Д. Булгантуяа) Оюутан: B241940061 (Л. Ариунзаяа) Оюутан: B242270131 (У. Янжинлхам) Оюутан: B242270045 (Ч. Мөнхбаяр) Оюутан: B180930028 (Б. Энхбаяр) Оюутан: B241910015 (Т. Цэгц)

Улаанбаатар · 2026 он

Хураангуй

UNIX Shell Explorer нь C хэл дээр бичсэн командын интерпретатор бөгөөд хэрэглэгчийн оруулсан командыг child процесст гүйцэтгэнэ. Сурах бичгийн дагуу процесс үүсгэх, команд ажиллуулах, history, background гүйцэтгэл, оролт/гаралтын чиглүүлэлт болон pipe IPC-ийг хэрэгжүүлсэн. Түүнчлэн Python локал сервер, HTML интерфэйсээр команд өгөх, гаралтыг харах, процессын PID ба төлөвийн үйл явдлыг модоор дүрслэх боломжтой. MacBook дээр хийсэн тестийн screenshot-д “PASS: 8 tests” гарсан.

1. Удиртгал ба зорилго

Энэ ажлын зорилго нь Operating System Concepts (10th edition) номын 3-р бүлгийн Programming Project 1 — UNIX Shell даалгаврыг гүйцэтгэж, процесс удирдах системийн дуудлагуудыг практикт туршиж ойлгох, хэрэглэгчийн оруулсан өөр өөр командыг ажиллуулах, процессын parent–child холбоог бодит PID-ээр харуулах явдал юм.

2. Сурах бичгийн шаардлага

Хүснэгт 2.1. Сурах бичгийн шаардлага ба хэрэгжүүлсэн арга

Шаардлага	**Хэрэгжүүлсэн арга**
Команд ба child процесс	`fork()`, `execvp()`, `waitpid()`
History	`!!` — өмнөх командыг давтах
Background	`&` — командын дуусахыг хүлээлгүй буцах
Оролт / гаралтын чиглүүлэлт	`open()` болон `dup2()`
Pipe IPC	`pipe()` ба `dup2()`, хоёр процессын хооронд
UI дүрслэл	PID, parent–child үйл явдлын бүртгэлээс process tree байгуулах

3. Онолын үндэс

Процесс нь ажиллаж буй программын гүйцэтгэлийн нэгж юм. Shell нь процессыг доорх системийн дуудлагуудаар удирдана.

Хүснэгт 3.1. Shell-д хэрэглэсэн системийн дуудлагууд

Системийн дуудлага	**Үүрэг**
`fork()`	Эцэг процессоос хүүхэд процесс үүсгэнэ.

`execvp()`	Хүүхэд процессын программыг хэрэглэгчийн өгсөн командаар солино.
`waitpid()`	Эцэг процесс хүүхэд процесс дуустал хүлээнэ.
`pipe()`	Хоёр процессын хооронд нэг чиглэлтэй byte stream үүсгэнэ.
`dup2()`	Стандарт оролт, гаралтыг файл эсвэл pipe руу дахин чиглүүлнэ.

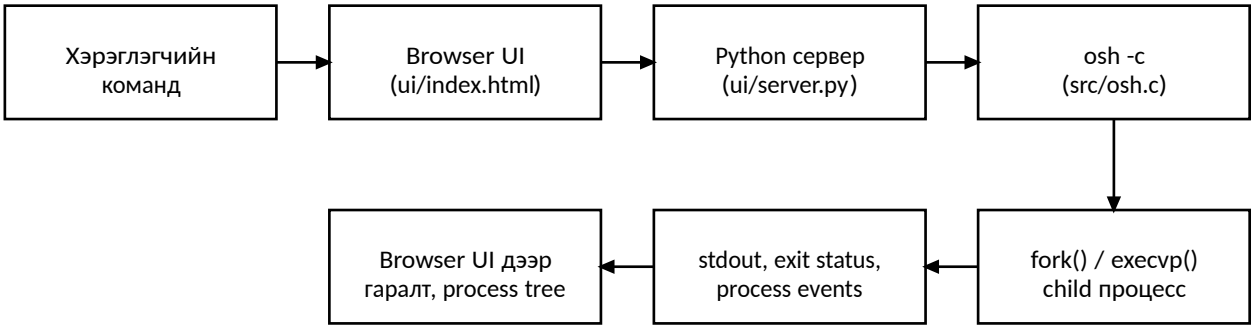
4. Системийн зохиомж

Хүснэгт 4.1. Системийн бүрэлдэхүүн

Бүрэлдэхүүн	**Файл**	**Үүрэг**
Frontend	`ui/index.html`	Команд оруулах талбар, гаралт, process tree, history харуулна.
Backend	`ui/server.py`	Локал HTTP сервер; C программыг ажиллуулж гаралт, event бүртгэлийг дамжуулна.
Shell	`src/osh.c`	Команд задлах, fork/exec, pipe, redirection хэрэгжүүлнэ.
Тест	`tests/test_shell.py`	Автомат тест.

Үйл явдлын файлд цаг, PID, PPID, state, command хадгалагдах бөгөөд UI үүнийг process tree болгон дүрслэнэ.

Өгөгдлийн урсгал. Хэрэглэгчийн команд → Browser UI → Python сервер → `osh -c` → fork/exec → stdout, exit status, process events → Browser UI.



Зураг 4.1. Системийн өгөгдлийн урсгал

Одоогийн хэрэгжилтэд команд бүрт шинэ `osh` процесс үүсдэг тул энэ нь нэг тасралтгүй shell session биш болно.

5. Хэрэгжилт ба орчин

Хэрэгжилтэд C11 (Clang/cc), Python 3, HTML/CSS/JavaScript ашигласан. Программыг macOS дээр VS Code болон Chrome ашиглан гараар ажиллуулж шалгасан.

Makefile-ийн compile команд: `cc -std=c11 -Wall -Wextra -O2 -o osh src/osh.c`

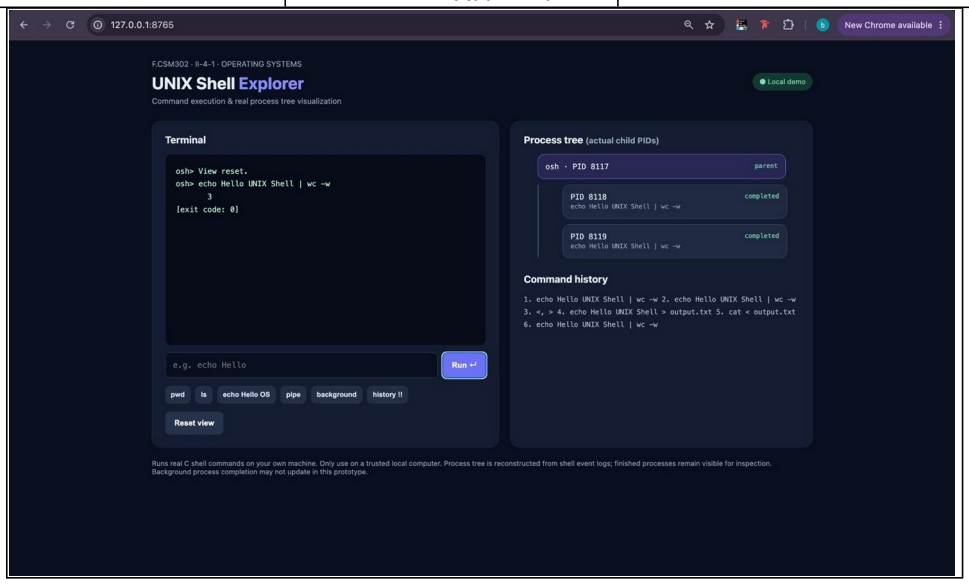
Веб UI-г `python3 ui/server.py` командаар асааж, `http://127.0.0.1:8765` хаягаар нээнэ.

6. Туршилтын арга ба үр дүн

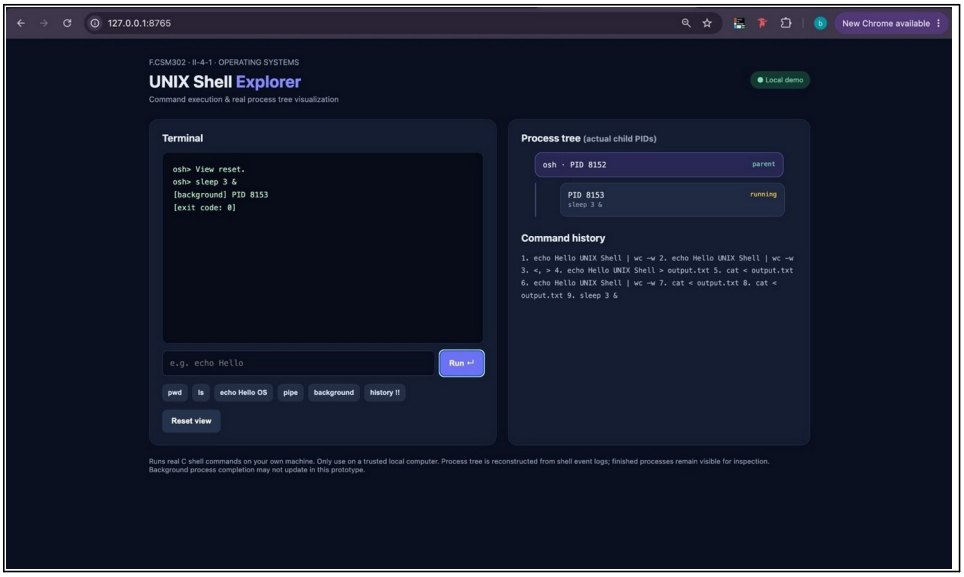
Туршилтыг багийн гишүүний MacBook дээрх macOS, VS Code Terminal, Chrome орчинд хийсэн. Хүснэгт 6.1-ийн ажиглалт нь доорх бодит screenshot болон терминалын үр дүнд үндэслэсэн.

Хүснэгт 6.1. Туршилтын үр дүн

Тест	Хүлээгдсэн үр дүн	Ажигласан үр дүн
`echo Hello UNIX Shell | wc -w`	3	3, exit code 0
`!!`	Өмнөх команд дахин ажиллана	Өмнөх команд дахин ажиллаж, history-д давтагдсан бичлэг нэмэгдсэн
`echo Hello UNIX Shell > output.txt`	output.txt файл үүснэ	exit code 0; гаралт файлд чиглэгдсэн
`cat < output.txt`	Hello UNIX Shell	Hello UNIX Shell гарсан (Хавсралт А)
`sleep 3 &`	PID үүсч, шууд буцна	Child PID 8153, төлөв running
`make test`	Автомат тестүүд тэнцэнэ	PASS: 8 tests



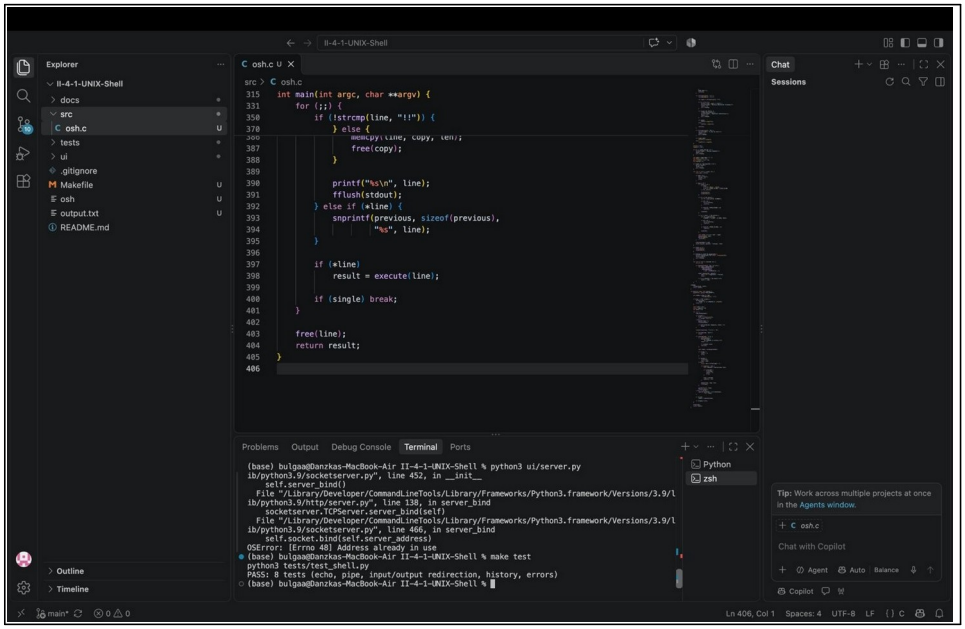
Зураг 6.1. Pipe ба history туршилт: гаралт 3, process tree



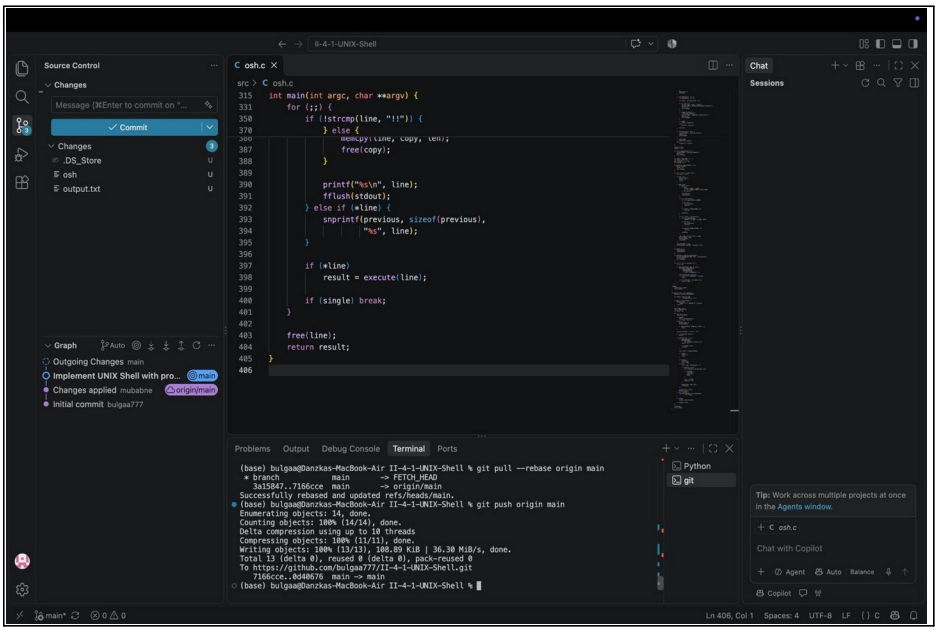
Зураг 6.2. Background: sleep 3 &, PID 8153 үүссэн, төлөв running

7. Автомат тест, GitHub нотолгоо

`make test` командаар ажиллах 8 автомат тест нь echo, pipe, оролт/гаралтын чиглүүлэлт, history болон алдааны тохиолдлыг шалгана. Эх код, тест, баримт бичгийг GitHub repository-д байршуулсан.



Зураг 7.1. make test: 8 автомат тест амжилттай



Зураг 7.2. git pull --rebase болон git push амжилттай

8. Хэлэлцүүлэг ба хязгаарлалт

- Команд бүрт шинэ `osh` (parent) процесс үүсдэг тул UI нь нэг тасралтгүй shell session биш.
- Process tree нь процессын event log-д суурилсан дүрслэл юм.
- Шинэчилсэн Python сервер background процессын PID-ийг шалгаж, running төлөвийг finished болгон шинэчилдэг болсон. Гэхдээ finished төлөв нь амжилттай exit code 0 гэдгийг батлахгүй.
- Parser нь сургалтын хялбар хувилбар тул олон pipe, shell expansion, бүрэн job control-ийг дэмжихгүй.
- 8/8 автомат тест нь бүх боломжит оролтын баталгаа биш.

9. Дүгнэлт

Сурах бичгийн үндсэн системийн дуудлага болон shell-ийн гол үйлдлүүдийг хэрэгжүүлж, бодит PID бүхий процессын дүрслэлтэй холбосон. Pipe, history, redirection болон background процессын үүсэлтийг туршсан. Нэмэлт засварын дараа background процесс дуусахад finished төлөв гардаг болсон (Хавсралт В). Цаашид нэг тасралтгүй shell session болон илүү найдвартай job tracking хэрэгжүүлэх шаардлагатай.

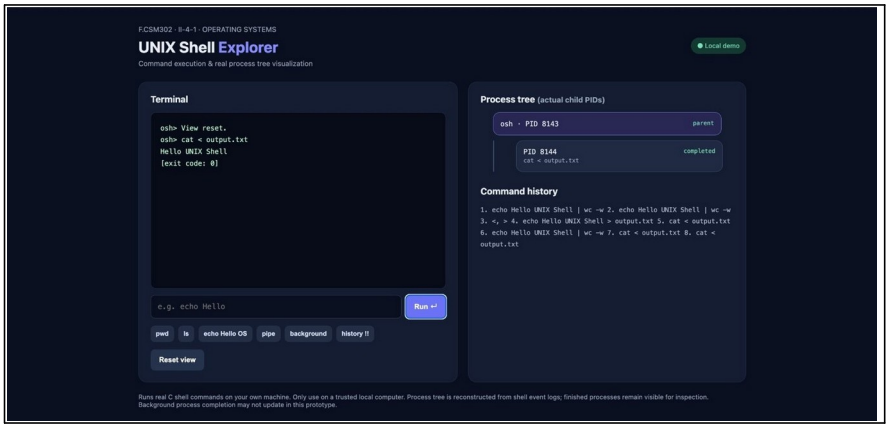
Ашигласан материал

1. 1F.CSM302 Үйлдлийн систем, Бие даалт 1 удирдамж, 2026.

2. F.CSM302, Сэдэв ба шүүмжийн хуваарилалт, II-4-1 UNIX Shell.

3. II-4-1-UNIX-Shell GitHub repository: source, tests, docs.

Хавсралт A. Input Redirection туршилт



Зураг A. cat < output.txt команд output.txt файлаас уншиж, гаралтыг харуулсан

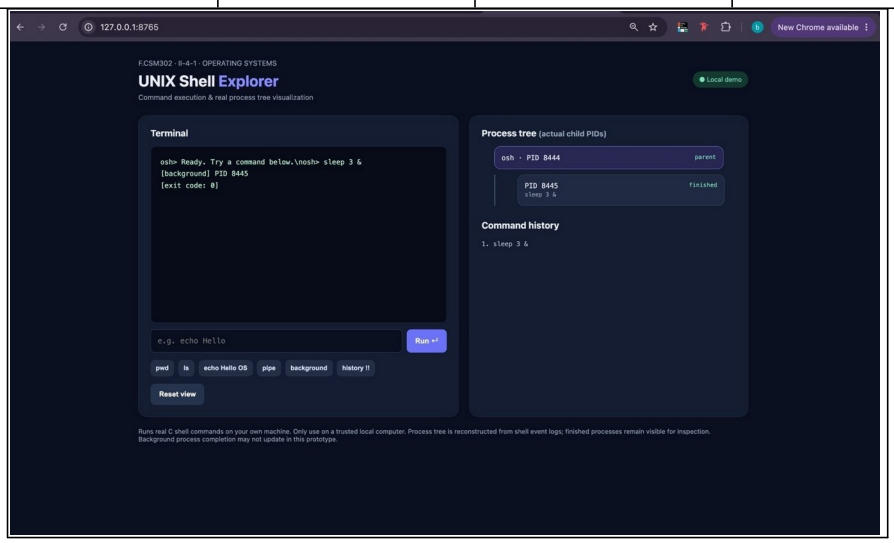
Ажигласан гаралт: Hello UNIX Shell, exit code 0.

Хавсралт B. Background төлөвийн шинэчлэл

Өмнөх зурагт (Зураг 6.2) child процесс running гэж харагдсан. Шинэчилсэн `ui/server.py` нь процесс дууссаныг шалгаж, дараагийн UI refresh-д finished төлөвөөр дүрсэлдэг. Үүнийг доорх бодит screenshot харуулна. Энэ screenshot нь туршилтыг дахин ажиллуулсан үеийнх тул PID нь Зураг 6.2-оос өөр байна.

Хүснэгт B.1. Background туршилтын PID

Туршилт	Parent PID	Child PID	Төлөв
`sleep 3 &`	8444	8445	finished



Зураг B. sleep 3 &: процесс дууссаны дараа Process Tree-д finished төлөв

Ажиглалт: Процесс дууссаны дараа Process Tree-д finished гэж харагдсан. **Тайлбар:** finished төлөв нь процесс цаашид ажиллахгүй байгааг илэрхийлнэ. Амжилттай дууссан эсэхийг exit status-аар тусад нь шалгана.